Configured MBOM Builder: Architecture

MCP server for Claude Desktop · October 16, 2026

The builder has three layers. **Claude Desktop** is the interface. A small **local MCP server** (Python) holds the logic. The **3DEXPERIENCE web services** are where everything gets created. An **Excel workbook** is the input that drives it all.

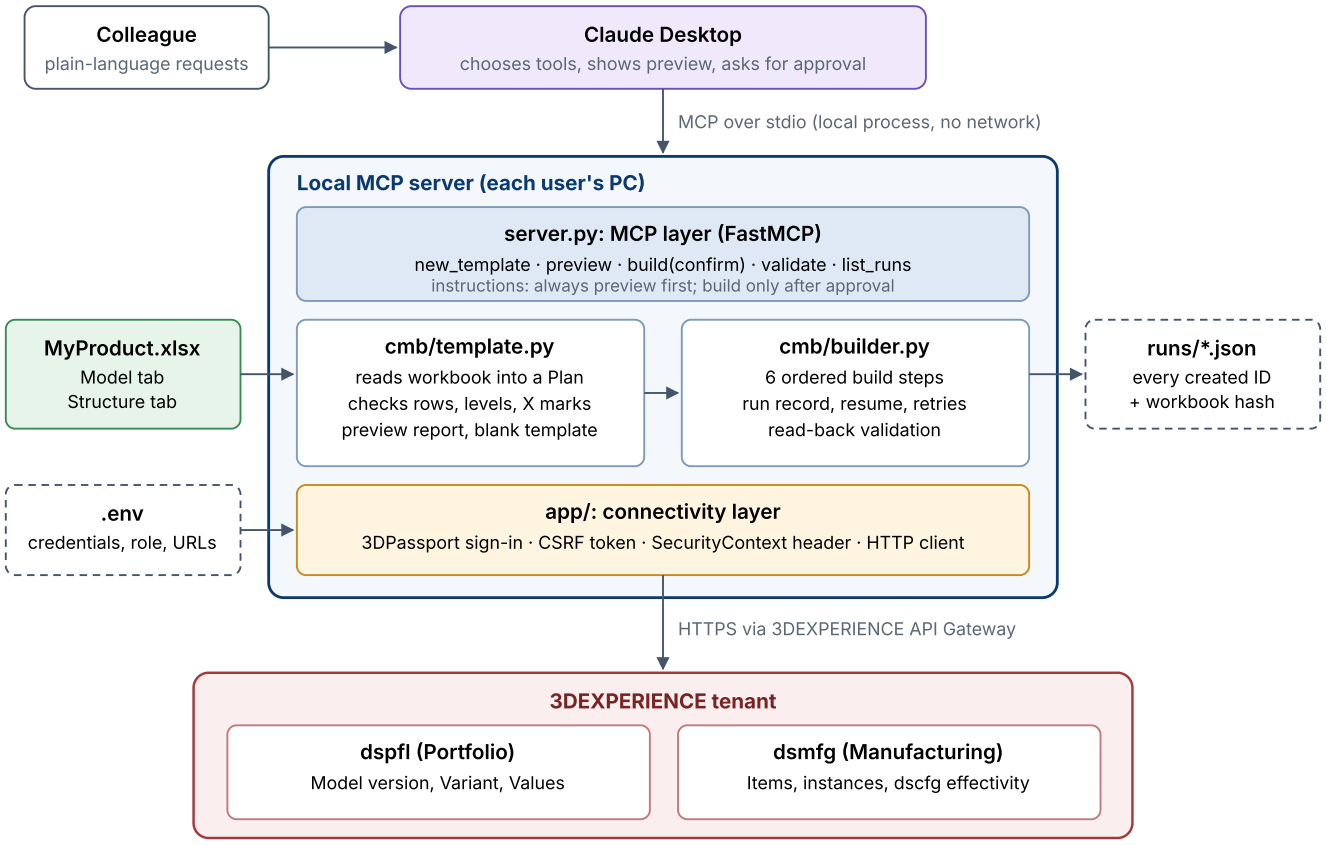


Figure 1. Components and data flow

The pieces

Layer	What it does
`server.py` MCP layer	A FastMCP server that Claude Desktop starts on its own using the config entry. It exposes five tools. Its built-in instructions tell Claude to always preview first and to build only with `confirm=true` after the user approves. It holds almost no logic itself and hands the work to `cmb/`.
`cmb/template.py` Input layer	Reads the workbook into a *Plan*: the model version, variant and values from the Model tab, and one *Line* per row of the Structure tab, with the X marks turned into effectivity. Checks the sheet: required fields, valid levels, quantities, X marks only on direct children of the configured item, and Existing item IDs. Also produces the preview report and generates the blank template.
`cmb/builder.py` Execution layer	Runs the build as six fixed steps (below). After every created object it writes the ID to a **run record** together with a hash of the workbook, which makes **resume** safe. It retries on 429/5xx errors and treats "HTTP 200 with an errorCode" as a failure, a quirk of the set/variant call.

`app/` Connectivity layer	A copy of the sign-in and HTTP client from tdx-mcp-server: signs in to 3DPassport, gets the CSRF token, adds the SecurityContext header and calls the API Gateway URLs from `.env`. It is the only part that knows how authentication works; if your admin requires a different sign-in method, this is the one folder to change.
Supporting	`templates/` (blank template, R648 example, mini test) · `tests/` (a mock tenant shaped on real responses, so the full build can be tested offline) · `install.ps1` (sets up the virtual environment and `.env`).

Build steps

#	Step	What happens	Web service
1	Portfolio	Model version (Hardware Product), variant and variant values; variant attached to the model version (constraint Must)	`dspfl` ModelVersion, Variant, VariantValue, VariantInstance
2	Items	Creates the new Manufacturing Items; reuses Existing ones	`dsmfg` MfgItem
3	Instances	Quantity N becomes N instances under each parent	`dsmfg` MfgItemInstance
4	Configuration	Attaches the model to the configured item and enables the Variant criterion (without it, effectivity fails)	`dscfg:Configured` attach + PATCH
5	Effectivity	Writes the CfgEffectivityExpression XML on each variant-specific instance; several values joined by OR	`dscfg:Filterable/set/variant`
6	Validate	Reads everything back and checks that each variant value resolves to the lines the sheet expects	`dsmfg` instance list, `dscfg:Filterable` read

How a request flows

1. The user says "preview C:\Work\X.xlsx" and Claude calls `preview_configured_mbom`.
2. `template.py` builds and checks the Plan; the builder checks the tenant (model title unused, Existing IDs exist).
3. Claude shows the plan and any problems.
4. The user approves and Claude calls `build_configured_mbom(confirm=true)`.
5. The builder runs the six steps, writing the run record as it goes.
6. It returns a summary with the validation result. If it stops part-way, the user fixes the cause and asks Claude to resume; anything already recorded is skipped.

Design choices worth knowing

- **Tools work at the level of the task, not the API.** Claude doesn't string together 50 web service calls itself; the builder does that the same way every time. That makes the result repeatable and much less dependent on how a colleague words the request.
- **It's separate from tdx-mcp-server.** It copies `app/` rather than depending on your setup, so colleagues only need this one folder and their own credentials.
- **It runs entirely on each user's PC.** There's no shared service, and each person's builds run under their own 3DEXPERIENCE role and permissions.
- **Sign-in caveat.** The data calls use Dassault's documented web services, but the sign-in imitates the 3DPassport browser login. It can break if that page changes and does not work on SSO-only tenants.